SEGURANÇA DE APLICAÇÕES | DEPENDÊNCIAS NPM

Relatório técnico de vulnerabilidades

Consolidação dos 10 alertas de segurança informados, com descrição técnica, impacto, evidências, alternativas e atualizações recomendadas.

Data de referência: 27 de outubro de 2026 • Ecossistema: npm • Idioma técnico original preservado nos detalhes

4	6	7
CRÍTICOS	**ALTOS**	**PACOTES / NOMES DISTINTOS**

Quadro consolidado de ações

Pacote	Versão	Sev.	CVE / GHSA	Atualizar para
DOMPurify	3.4.14	Crítico	GHSA-cg2g-xh64-v4q7	3.4.17
proxy-addr	2.0.7	Crítico	CVE-2026-90711	2.0.8
jsPDF	3.0.4	Crítico	CVE-2026-31938	4.2.1
jsPDF	3.0.4	Crítico	CVE-2025-68428	4.0.0*
body-parser	1.20.6	Alto	CVE-2026-111181	1.20.9
DOMPurify	3.4.14	Alto	GHSA-mrv2-3pqr-j5jr	3.4.17
vite	6.4.3	Alto	CVE-2026-112942	6.4.4
@fastify/busboy	3.2.0	Alto	CVE-2026-19481	3.2.1
@fastify/busboy	3.2.0	Alto	CVE-2026-19484	3.2.1
@grpc/grpc-js	1.14.4	Alto	CVE-2026-101916	1.14.5

* jsPDF 4.0.0 corrige especificamente CVE-2025-68428; para contemplar também CVE-2026-31938, a recomendação consolidada é 4.2.1 ou superior, sujeita aos testes de compatibilidade.

Nota de consistência e atualização

O texto original do primeiro alerta DOMPurify apresenta o status histórico "not fixed" e cita 3.4.16 como última versão. A base pública atual indica correção em 3.4.17. Para Vite 6.4.3, a ação genérica "Ajustar" foi detalhada com a correção 6.4.4 para o ramo 6.x.

Escopo: este material reproduz e organiza as informações disponibilizadas. As classificações Crítico/Alto seguem a lista fornecida, não substituem a avaliação contextual de risco e não comprovam exploração no ambiente da organização.

Orientação prática de remediação

• Atualizar preferencialmente os pacotes por meio do gerenciador de dependências (package-lock, pnpm-lock ou yarn.lock), incluindo dependências transitivas.

• Consolidar correções quando há múltiplas vulnerabilidades no mesmo pacote: DOMPurify 3.4.17; jsPDF 4.2.1; @fastify/busboy 3.2.1.

• Aplicar correção do Vite 6.4.3 para 6.4.4 ou outra versão corrigida compatível com o ramo utilizado. Verificar uso do launch-editor e exposição de funcionalidades de desenvolvimento em Windows.

• Executar testes funcionais e de regressão em geração de PDFs, validação de HTML, manipulação de proxies, requisições multipart e comunicação gRPC.

• Validar o inventário e a resolução efetiva das versões após o deploy (npm ls, SBOM, varredura de dependências e nova análise de alertas).

• Registrar evidências, responsável técnico, data de correção e riscos residuais antes do encerramento dos alertas.

Legenda do relatório

Os trechos de código são provas de conceito e exemplos técnicos fornecidos no texto de origem, mantidos para documentação e revisão por profissionais autorizados. Não representam testes realizados neste relatório.

Seções em inglês foram preservadas na linguagem técnica original para evitar perda de precisão nas condições de exploração, identificadores e exemplos de código.

dompurify @ 3.4.14

SAFE_FOR_TEMPLATES bypass: split template expressions survive in attached shadow root content on DOM output modes

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
GHSA-cg2g-xh64-v4q7	22/10/2026	**Upgrade para 3.4.17**

Summary

When DOMPurify is configured with SAFE_FOR_TEMPLATES: true together with a DOM-returning mode (RETURN_DOM: true, RETURN_DOM_FRAGMENT: true, or IN_PLACE: true), template expressions such as {{...}}, ${...}, and <%...%> survive sanitization inside the text content of an attached shadow root. The post-walk safety net _scrubTemplateExpressions (src/purify.ts:1717) was extended to recurse into <template>.content, but it never recurses into attached shadow roots. An attacker who can supply a DOM node carrying a shadow root (the IN_PLACE and shadow-aware RETURN_DOM surfaces) can split an expression across text nodes separated by disallowed elements; after those elements are removed the fragments become adjacent and serialize into a complete expression that a downstream template engine will evaluate.

Root Cause

SAFE_FOR_TEMPLATES neutralizes template expressions through two mechanisms: a per-text-node strip during the main walk (_stripTemplateExpressions, applied in _sanitizeElements at src/purify.ts:2246), and a final post-walk pass (_scrubTemplateExpressions, src/purify.ts:1717) that normalizes adjacent text nodes and re-strips expressions that only form after merging. The per-node strip cannot catch an expression split into individual fragments, because each fragment fails the delimiter regexes. The post-walk pass is the only mechanism that would catch the merged form, but it is structurally unable to reach shadow root content.

```
_scrubTemplateExpressions (src/purify.ts:1717-1751):
  - node.normalize() at line 1718 does not merge text nodes inside shadow roots
    (DOM spec).
  - NodeIterator at lines 1723-1731 does not descend into shadow trees
    (DOM spec).
  - Explicit recursion at lines 1743-1750 enters <template>.content only,
    via querySelectorAll('template'); there is no equivalent recursion into
    attached shadow roots.
Shadow roots are sanitized earlier by _sanitizeShadowDOM (src/purify.ts:2710),
which applies the per-node strip but never performs the post-walk
normalize-and-rescrub.
```

Shadow root text nodes are therefore sanitized only per-node and never re-examined as a merged unit.

Impact

An attacker who controls DOM-node input containing an attached shadow root can inject template expressions that survive sanitization and are later evaluated by the application's template engine. In client-side contexts this yields cross-site scripting; in a server-side template context it can escalate to template injection. Exploitation requires non-default application configuration: the application must enable SAFE_FOR_TEMPLATES and a DOM-returning mode, pass a DOM node that already has an attached shadow root, and feed the sanitized shadow root content into a template evaluator. The shadow root does not have to be built programmatically with attachShadow: in engines that support declarative shadow DOM, attacker-supplied markup containing <template shadowrootmode="open"> parsed through setHTMLUnsafe or Document.parseHTMLUnsafe produces the same attached shadow root, so a plain HTML string reaches the gap once the application is configured this way. Demonstrated capability is injection of a complete, attacker-chosen expression into post-sanitization shadow root content; the resulting script execution and any confidentiality or integrity consequence depend on the downstream engine's evaluation context.

Proof of Concept - Preconditions

```
- DOMPurify built from the affected source (dist/purify.js or ESM/CJS bundle).
- A browser (or DOM implementation) with shadow DOM support.
- Application constructs a DOM node with an attached shadow root before
  sanitizing and later evaluates the shadow root text as a template.
```

Build a host element whose shadow root holds a template expression split across text nodes separated by disallowed elements, then sanitize in place with SAFE_FOR_TEMPLATES:

```
const host = document.createElement('div');
const root = host.attachShadow({ mode: 'open' });

// Split "${alert(document.domain)}" so no single fragment matches a regex.
// <object> is not in default ALLOWED_TAGS and is removed during sanitization.
root.appendChild(document.createTextNode('$'));
root.appendChild(document.createElement('object'));
root.appendChild(document.createTextNode('{alert(document.domain)}'));

DOMPurify.sanitize(host, { IN_PLACE: true, SAFE_FOR_TEMPLATES: true });

host.shadowRoot.normalize();
console.log(host.shadowRoot.textContent);

${alert(document.domain)}
```

A downstream template evaluator reading shadowRoot.textContent (for example new Function('return `' + content + '`')() or any ${...}/{{...}} engine) then evaluates the fully formed expression.

Attack Chain

• 1. Exposure: The application calls DOMPurify.sanitize(node, cfg) (src/purify.ts:2854) with SAFE_FOR_TEMPLATES: true and IN_PLACE: true (or a shadow-aware RETURN_DOM), passing a DOM node that already carries an attached shadow root.

• 2. Control: The attacker controls the shadow root text nodes and the disallowed separator elements between them; each fragment ($, {alert(document.domain)}) is an individually benign text node.

• 3. Path: _sanitizeAttachedShadowRoots (src/purify.ts:2977) feeds the root into _sanitizeShadowDOM (src/purify.ts:2710), which strips expressions per text node (src/purify.ts:2246) and removes separators.

• 4. Guard: The post-walk safety net _scrubTemplateExpressions (src/purify.ts:3131-3132) is the only pass that would catch the merged expression, but it never recurses into shadow roots (src/purify.ts:1717-1751 enters <template>.content only).

• 5. Primitive: Adjacent fragments serialize as a single concatenated string on shadowRoot.textContent / shadowRoot.innerHTML, reconstituting the complete expression.

• 6. Result: A downstream template engine evaluates the attacker-chosen expression, achieving script execution or template injection in the engine context.

Bypass Evidence

The relevant protection is SAFE_FOR_TEMPLATES, specifically the post-walk pass that defends against expressions that only form after adjacent text nodes merge. That defense was previously extended to <template>.content under CVE-2026-65900 (GHSA-gvmj-g25r-r7wr) by adding querySelectorAll('template') recursion at src/purify.ts:1743-1750. Enumerating every call site of _scrubTemplateExpressions (src/purify.ts:1717, 1747, 3132, 3141) shows none receive a shadow root, and _sanitizeShadowDOM (src/purify.ts:2710) never calls _scrubTemplateExpressions or normalize(). The per-node strip cannot compensate: MUSTACHE_EXPR (src/regexp.ts:3) requires {{ or }}, TMPLIT_EXPR (src/regexp.ts:5) requires ${, and ERB_EXPR (src/regexp.ts:4) requires <% or %>, none of which appear in a single-character fragment. The CVE-2026-65900 regression tests exercise <template>.content only; no test covers shadow root content under SAFE_FOR_TEMPLATES. The gap is therefore an unfixed case on a distinct remediation boundary (shadow roots) rather than a bypass of the delivered template-content fix.

Affected Versions - texto de origem

• Ecosystem: npm

• Package: dompurify

• Confirmed affected range: >= 3.4.4, <= 3.4.16

• Latest release checked in the original narrative: 3.4.16

• Historical fix status stated in original narrative: not fixed

The lower bound 3.4.4 is the release that introduced both the attached-shadow-root sanitization pass and the _scrubTemplateExpressions safety net; releases 3.4.4 through 3.4.7 carry the safety net with no content-container recursion, and 3.4.8 onward add <template>.content recursion but still omit shadow root recursion. The original narrative stated that no published release and no development revision contained a shadow-root fix.

Atualização de status em 27/10/2026

O aviso atual consultado lista 3.4.17 como patched version. Assim, a frase original "not fixed" não deve ser interpretada como estado atual.

Suggested Fix

Enforce the same post-walk normalize-and-rescrub invariant on attached shadow roots that already applies to <template>.content. After _sanitizeShadowDOM finishes walking a shadow root, run _scrubTemplateExpressions over that shadow root so adjacent text nodes created by element removal are merged and re-stripped; alternatively, have _scrubTemplateExpressions discover and recurse into attached shadow roots as it does for template content. The fix must cover nested shadow roots and shadow roots inside <template>.content, and it must handle all three expression dialects, since any disallowed element can serve as the separator that produces adjacent fragments.

Reported by zx (GitHub: @manus-pi).

proxy-addr @ 2.0.7

IP spoofing via IPv4-mapped IPv6 trust subnet

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-90711	05/10/2026	**Upgrade para 2.0.8**

Impact

proxy-addr determines which network hops are trusted proxies so that X-Forwarded-For can be believed. When an application configures a trust subnet as an IPv4-mapped IPv6 address with a short prefix, such as ::ffff:10.0.0.0/8 (the correct spelling is ::ffff:10.0.0.0/104), the subnet compiles with all-zero leading bits and matches every IPv4 address instead of the block it names. The same happens for any IPv6 trust subnet with zero leading bits, such as ::/1.

Every unauthenticated client is then trusted as a proxy at hop 0, so proxyaddr(req, trust), and therefore req.ip and req.ips in Express, returns whatever the client sends in X-Forwarded-For. This defeats IP-based access control, rate limiting, geolocation, and audit logging. The misconfiguration compiles without any error, and the same block written correctly behaves correctly, so the difference is not visible from the configuration.

Patches

Upgrade to proxy-addr 2.0.8. An IPv4 address now matches an IPv6 trust subnet only when that subnet is a genuine IPv4-mapped subnet whose prefix covers the mapped marker.

Workarounds

Write IPv4 trust subnets in plain IPv4 notation (for example 10.0.0.0/8). If IPv4-mapped IPv6 notation is required, use the full form so the prefix covers the mapped marker (for example ::ffff:10.0.0.0/104 for the 10.0.0.0/8 block).

jspdf @ 3.0.4

jsPDF has HTML Injection in New Window paths

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-31938	17/03/2026	**Upgrade para 4.2.1**

Impact

User control of the options argument of the output function allows attackers to inject arbitrary HTML (such as scripts) into the browser context the created PDF is opened in. The affected overloads and options are:

• "pdfobjectnewwindow": the pdfObjectUrl option and the entire options object, which is JSON-serialized and included verbatim in the generated HTML-string.

• "pdfjsnewwindow": the pdfJsUrl and filename options.

• "dataurlnewwindow": the filename option.

The vulnerability can be exploited in the following scenario: the attacker provides values for the output options, for example via a web interface. These values are then passed unsanitized (automatically or semi-automatically) to the attack victim. The victim creates and opens a PDF with the attack vector using one of the vulnerable method overloads inside their browser. The attacker can thus inject scripts that run in the victim browser context and can extract or modify secrets from this context.

Example attack vector

```
import { jsPDF } from 'jspdf';
const doc = new jsPDF();

const payload = 'x\"></iframe><script>window.__n=1</script><iframe src="';

doc.output('pdfjsnewwindow', {
  filename: payload,
  pdfJsUrl: 'viewer.html'
});
```

Patches

The vulnerability has been fixed in jspdf@4.2.1.

Workarounds

Sanitize user input before passing it to the output method.

Additional metadata provided: severity 9.6 / Critical; CWE-79; affected versions < 4.2.1.

jspdf @ 3.0.4

jsPDF has Local File Inclusion / Path Traversal vulnerability

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2025-68428	05/01/2026	**Upgrade para 4.0.0 (preferir 4.2.1 para cobrir ambas)**

Impact

User control of the first argument of the loadFile method in the node.js build allows local file inclusion/path traversal. If given the possibility to pass unsanitized paths to the loadFile method, a user can retrieve file contents of arbitrary files in the local file system the node process is running in. The file contents are included verbatim in the generated PDFs.

Other affected methods are: addImage, html, addFont.

Only the node.js builds of the library are affected, namely the dist/jspdf.node.js and dist/jspdf.node.min.js files.

Example attack vector

```
import { jsPDF } from "./dist/jspdf.node.js";

const doc = new jsPDF();

doc.addImage("./secret.txt", "JPEG", 0, 0, 10, 10);
doc.save("test.pdf"); // the generated PDF will contain secret.txt
```

Patches

The vulnerability has been fixed in jsPDF@4.0.0. This version restricts file system access by default. The original advisory states this semver-major update does not introduce other breaking changes.

Workarounds

With recent node versions, jsPDF recommends using the --permission flag in production. The feature was introduced experimentally in v20.0.0 and is stable since v22.13.0/v23.5.0/v24.0.0. See the Node.js documentation: https://nodejs.org/api/permissions.html . For older node versions, sanitize user-provided paths before passing them to jsPDF.

Credits: Researcher kilkat (Kwangwoon Kim).

body-parser @ 1.20.6

Denial of Service via trailing bytes after a compressed request body

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-111181	23/10/2026	**Upgrade para 1.20.9 (ou 2.4.0)**

Impact

A single request can crash any application using body-parser with default options. When a request carries a Content-Encoding (for example gzip), body-parser pipes it into a decompression stream and, after reading the body, leaves that stream piped with no error listener. A payload of one valid compressed frame followed by trailing bytes decodes and the application responds normally, but the trailing bytes then reach the already-finished decoder, which emits an error that is re-thrown as an uncaught exception and terminates the process. No authentication or matching route is required, since parsing runs before routing.

Patches

Fixed in body-parser 2.4.0 and 1.20.9. Upgrade to one of these versions.

Workarounds

Pass inflate: false to the body-parser middleware if the application does not need to accept compressed request bodies. This rejects encoded requests before any decompression stream is created.

dompurify @ 3.4.14

IN_PLACE hook removes a parent or sibling, leaving detached subtree event handlers active

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
GHSA-mrv2-3pqr-j5jr	22/10/2026	**Upgrade para 3.4.17**

Summary

The 3.4.16 fix for GHSA-p98j-92pf-mc4p only covers a hook that removes the node it was called for. If an IN_PLACE hook removes a different node, like the current node parent or a sibling that has not been visited yet, that subtree is never walked or neutralized. Event handlers in it stay on the elements, and an <img onerror> that was already loading fires after sanitize() returns.

The README says: "Nodes that a hook detaches from the tree ... are treated as removed. In IN_PLACE mode their subtree is neutralized inline, so an <img onload> that was already loading when you built the live tree cannot fire after sanitize() returns." That does not hold when the detached node is not the hook own node.

Tested versions

3.4.16 (latest at the time of testing) and main at 18f8f0e; also 3.4.15, 3.4.12 and 2.5.8. jsdom used for the matrix below, Chromium (Playwright) for the actual event firing.

Details

At src/purify.ts line 2134 (dist/purify.cjs.js line 2009 in 3.4.16), _handleHookDetachedNode() contains:

```
if (currentNode === root || getParentNode(currentNode) !== null) {
  return false;
}
if (IN_PLACE) {
  _neutralizeSubtree(currentNode);
}
```

It only checks whether currentNode lost its parent. When a hook removes currentNode.parentNode (or any ancestor), currentNode still has a parent, so nothing happens. The rest of the removed subtree is no longer reachable by the walker, and it is not in DOMPurify.removed, so the post-walk pass does not see it either. The same happens when a hook removes a later sibling. This applies to all five element and attribute hooks.

PoC (jsdom)

```
const { JSDOM } = require('jsdom')
const createDOMPurify = require('dompurify')
const win = new JSDOM('<!doctype html><body></body>').window
const doc = win.document

for (const mode of ['self', 'parent', 'nextSibling']) {
  const DOMPurify = createDOMPurify(win)
  doc.body.innerHTML = '<div id=root><figure><img class=bad src=a.png>' +
    '<img id=evil src=x onerror=alert(1)></figure><p>ok</p></div>'
  const root = doc.getElementById('root')
  const evil = doc.getElementById('evil')

  DOMPurify.addHook('afterSanitizeAttributes', (node) => {
    if (node.getAttribute && node.getAttribute('class') === 'bad') {
      if (mode === 'self') node.remove()
      if (mode === 'parent') node.parentNode.remove()
      if (mode === 'nextSibling') node.nextElementSibling.remove()
    }
  })

  DOMPurify.sanitize(root, { IN_PLACE: true })
  console.log(DOMPurify.version, mode.padEnd(11),
              'onerror =', JSON.stringify(evil.getAttribute('onerror')))
}
```

Output on 3.4.16

```
3.4.16 self        onerror = null
3.4.16 parent      onerror = "alert(1)"
3.4.16 nextSibling onerror = "alert(1)"
```

In self mode the walker still reaches the second image and strips onerror. In the other two modes the second image is detached with its handler still set. The same result is achieved with each of beforeSanitizeElements, uponSanitizeElement, afterSanitizeElements, beforeSanitizeAttributes and afterSanitizeAttributes.

PoC (browser)

The same thing in a real page, served from 127.0.0.1 and opened in Chromium:

```
<!doctype html><meta charset=utf-8>
<script src="purify.min.js"></script>
<body><pre id=log></pre>
<script>
window.fired = [];
function run(mode) {
  const D = DOMPurify(window);
  D.addHook('afterSanitizeAttributes', n => {
    if (n.getAttribute && n.getAttribute('class') === 'bad') {
      if (mode === 'parent') n.parentNode.remove();
      if (mode === 'self') n.remove();
    }
  });
  const root = document.createElement('div');
  document.body.appendChild(root);
  root.innerHTML = '<figure><img class=bad src=data:,><img src=x ' +
     'onerror="fired.push(\'' + mode + '\')"></figure><p>ok</p>';
  D.sanitize(root, { IN_PLACE: true });
  return root.outerHTML;
}
const a = run('self'), b = run('parent');
setTimeout(() => {
  document.getElementById('log').textContent =
    'version=' + DOMPurify.version + '\nself out=' + a +
    '\nparent out=' + b + '\nfired=' + JSON.stringify(fired);
}, 1500);
</script>
```

Result on 3.4.16

```
version=3.4.16
self out=<div><figure><img src="x"></figure><p>ok</p></div>
parent out=<div><p>ok</p></div>
fired=["parent"]
```

The returned tree is clean in both cases, but the handler on the detached image runs in the page for parent.

Impact

An app that uses IN_PLACE with a hook that removes a wrapper or a nearby element (for example "drop the whole <figure> if its image is not allowed") gets DOM XSS from attacker markup after sanitize() returns. It needs that kind of hook, so it falls under "input plus an application choice" in SECURITY.md, but it breaks the guarantee the README gives for hook detached nodes and is the same pattern as GHSA-55q2-fjhq-7xh7 and GHSA-p98j-92pf-mc4p.

Suggested Fix

Instead of only checking currentNode, check after each hook call whether anything outside the current node was detached. Two simple options:

• Before calling a hook in IN_PLACE mode, remember the next node the walker will visit and the current node ancestors; after the hook, neutralize any of them that are no longer attached under root.

• After the walk, neutralize every element that was in the original tree but is no longer connected to root (for example by collecting the elements before the walk).

vite @ 6.4.3

launch-editor: fs.existsSync() existence check can be bypassed on Windows

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-112942	20/10/2026	**Ajustar / upgrade do ramo 6.x para 6.4.4**

Summary

launch-editor checked fs.existsSync(fileName) before passing a supplied path to an editor command. On Windows, an attacker can bypass this check with a crafted path ending in \..:line:column. This removes the requirement that the attacker be able to place a file with a malicious filename.

Combined with the % escaping issue described in GHSA-698v-4r3x-3vcv, this bypass can lead to OS command injection when an attacker controls the file argument.

Impact

Only applications that run on Windows are affected.

Remediação contextualizada

A instrução original é "Ajustar". A atualização pública identifica Vite 6.4.4 como a versão corrigida mais próxima da versão 6.4.3 informada. Outras linhas corrigidas incluem 7.3.7, 8.1.6, 8.2.4 e 8.3.3; para launch-editor isolado, versão 2.14.2.

@fastify/busboy @ 3.2.0

Denial of Service via prototype-named multipart part header

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-19481	02/10/2026	**Upgrade para 3.2.1**

Impact

Versions of @fastify/busboy from 1.0.0 and prior to 3.2.1 are vulnerable to a Denial of Service. The multipart header parser stores part-header names on a plain JavaScript object, so a part header named __proto__ or constructor resolves to an inherited value that is not an array, and the parser throws TypeError: this.header[h].push is not a function. Through the documented req.pipe(busboy) integration this surfaces as an error event, while direct write()/end() usage throws synchronously and can terminate the Node.js process if uncaught. The parser runs before application middleware, so any unauthenticated client that can submit multipart/form-data is affected.

Patches

Fixed in version 3.2.1.

Workarounds

Attach an error listener to the Busboy stream so the parser failure is handled rather than crashing the process, and wrap direct write()/end() calls in a try/catch. Upgrading to 3.2.1 removes the failure entirely.

@fastify/busboy @ 3.2.0

Denial of Service via oversized multipart boundary

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-19484	02/10/2026	**Upgrade para 3.2.1**

Impact

Versions of @fastify/busboy from 3.1.0 and prior to 3.2.1 are vulnerable to a Denial of Service. The vendored streaming multipart search stores its default skip distance in a Uint8Array(256). A multipart boundary of exactly 252 bytes makes the search needle 256 bytes, and the table entry wraps to zero, so a crafted request keeps the search in a CPU-bound loop and stalls the Node.js event loop. An unauthenticated client can trigger this with a single small request. Applications that use @fastify/busboy to parse multipart/form-data, directly or through @fastify/multipart, are affected.

Patches

Fixed in version 3.2.1.

Workarounds

Validate the multipart boundary before parsing and reject any boundary longer than the RFC 2046 limit of 70 characters (for example at a reverse proxy or in an onRequest hook). Upgrading to 3.2.1 removes the issue.

@grpc/grpc-js @ 1.14.4

getAuthContext can return unauthorized certificates as though authorized

IDENTIFICADOR	PUBLICADO / ATUALIZADO	REMEDIAÇÃO
CVE-2026-101916	30/09/2026	**Upgrade para 1.14.5 ou posterior**

Impact

When server credentials are created with the requireClientCertificate option set to false, getAuthContext does not distinguish between authorized and unauthorized certificates in its return value. This can create improper authentication vulnerabilities for @grpc/grpc-js users who use the result of getAuthContext for authentication.

In particular, @grpc/grpc-js-xds can both set the requireClientCertificate option to false and use the return value of getAuthContext for RBAC authentication in some configurations.

Patches

This vulnerability is fixed in 1.13.6 and 1.14.5.

Workarounds

@grpc/grpc-js users using getAuthContext this way can avoid this problem by setting requireClientCertificate to true. @grpc/grpc-js-xds users using RBAC can avoid this by setting the require_client_certificate field to true in the DownstreamTlsContext in the xDS configuration.

Additional metadata provided: severity 7.4 / High; CWE-295; affected versions < 1.14.5 in the 1.14.x line.

Referências e observações de verificação

As descrições técnicas e provas de conceito foram fornecidas pelo solicitante. Foram consultados registros públicos para confirmar as correções de DOMPurify, proxy-addr, body-parser, Vite e @fastify/busboy em 27/10/2026; os demais dados são reproduzidos da listagem original.

- DOMPurify GHSA-cg2g-xh64-v4q7
- DOMPurify GHSA-mrv2-3pqr-j5jr
- proxy-addr CVE-2026-90711
- body-parser CVE-2026-111181
- Vite CVE-2026-112942
- @fastify/busboy CVE-2026-19481
- @fastify/busboy CVE-2026-19484
- Registro NPM DOMPurify 3.4.16
- Node.js permission model

Ressalvas de aplicação

A atualização de pacotes depende de compatibilidade com a aplicação, verificação de bibliotecas transitivas e execução de testes. Vulnerabilidades reportadas em bibliotecas instaladas não demonstram, isoladamente, que a aplicação seja explorável. A criticidade final deve considerar configurações de uso, exposição e controles existentes.